Comments in Python

There are two types of comments in Python: single line comments and multi line comments.

Single line comments

These comments start with a hash symbol (#) and are useful to mention that the entire line till the end should be treated as comment. For example,

```
# To find sum of two numbers
a = 10  # store 10 into variable a
```

Here, the first line is starting with a # and hence the entire line is treated as a comment. In the second line, a = 10 is a statement. After this statement, # symbol starts the comment describing that the value 10 is stored into variable 'a'. The part of this line starting from # symbol to the end is treated as a comment.

Comments are non-executable statements. It means neither the Python compiler nor the PVM will execute them. Comments are for the purpose of understanding human beings and not for the compiler or PVM. Hence, they are called non-executable statements.

Multi line comments

When we want to mark several lines as comment, then writing # symbol in the beginning of every line will be a tedious job. For example,

```
# This is a program to find net salary of an employee
# based on the basic salary, provident fund, house rent allowance,
# dearness allowance and income tax.
```

Instead of starting every line with # symbol, we can write the previous block of code inside """ (triple double quotes) or ''' (triple single quotes) in the beginning and ending of the block as:

```
"""
This is a program to find net salary of an employee
based on the basic salary, provident fund, house rent allowance,
dearness allowance and income tax.
"""
```

Or,

```
'''
This is a program to find net salary of an employee
based on the basic salary, provident fund, house rent allowance,
dearness allowance and income tax.
'''
```

The triple double quotes (""") or triple single quotes (''') are called 'multi line comments' or 'block comments'. They are used to enclose a block of lines as comments.

Docstrings

In fact, Python supports only single line comments. Multi line comments are not available in Python. The triple double quotes or triple single quotes are actually not multi line comments but they are regular strings with the exception that they can span multiple lines. That means memory will be allocated to these strings internally. If these strings are not assigned to any variable, then they are removed from memory by the garbage collector and hence these can be used as comments.

So, using """ are ''' or not recommended for comments by Python people since they internally occupy memory and would waste time of the interpreter since the interpreter has to check them.

If we write strings inside """ or ''' and if these strings are written as first statements in a module, function, class or a method, then these strings are called *documentation strings* or *docstrings*. These docstrings are useful to create an API documentation file from a Python program. An API (Application Programming Interface) documentation file is a text file or html file that contains description of all the features of software, language or a product. When a new software is created, it is the duty of the developers to describe all the classes, modules, functions, etc. which are written in that software so that the user will be able to understand the software and use it in a proper way. These descriptions are provided in a separate file either as a text file or an html file. So, we can understand that the API documentation file is like a help file for the end user.

Let's take an example to understand how to create an API documentation file. The following Python program contains two simple functions. The first function is add() function that takes two numbers and displays their sum. The second one is message() function that displays a message "Welcome to Python". A function contains a group of statements and intends to perform a specific task. In Python, a function should start with the 'def' keyword. After that, we should write the function name along with parentheses as def add(). We will learn about functions fully in a later chapter.

Observe that the add(x,y) function calls a print() function that displays the sum of x and y. But it also contains documentation strings inside a pair of """. Similarly, the message() function which is the second function calls a print() function that displays the message "Welcome to Python". This function also contains documentation strings written inside a pair of '''. At the end of this program, we are calling these two functions. While calling the first function, we are passing two values 10 and 25 and calling it as add(10, 25). While calling the second function, we are not passing any value. We are calling it as message() only. Now, consider the following program:

```
# program with two functions
def add(x, y):
    """
    This function takes two numbers and finds their sum.
    It displays the sum as result.
    """
    print("Sum= ", (x+y))
```

```
    def message():
        '''
        This function displays a message.
        This is a welcome message to the user.
        '''
        print("Welcome to Python")

    # now call the functions
    add(10, 25)
    message()
```

After typing this program, save it as ex.py and then execute it as:

```
    F:\PY>python ex.py
    Sum= 35
    Welcome to Python
```

This is the general way of executing any Python program. Now, we will execute this program once again to create API documentation file. For this purpose, we need a module *pydoc*. In the following command, we are using –m to represent a module and pydoc is the module name that is used by the python interpreter. After that –w indicates that an html file is to be created. The last word 'ex' represents the source file name.

```
    F:\PY>python  -m  pydoc  -w  ex
    Sum= 35
    Welcome to Python
    Wrote to ex.html
```

As understandable from the above lines, the output of the program is displayed and also the ex.html file is created. Open the ex.html file in any Web browser, as shown in Figure 3.1:

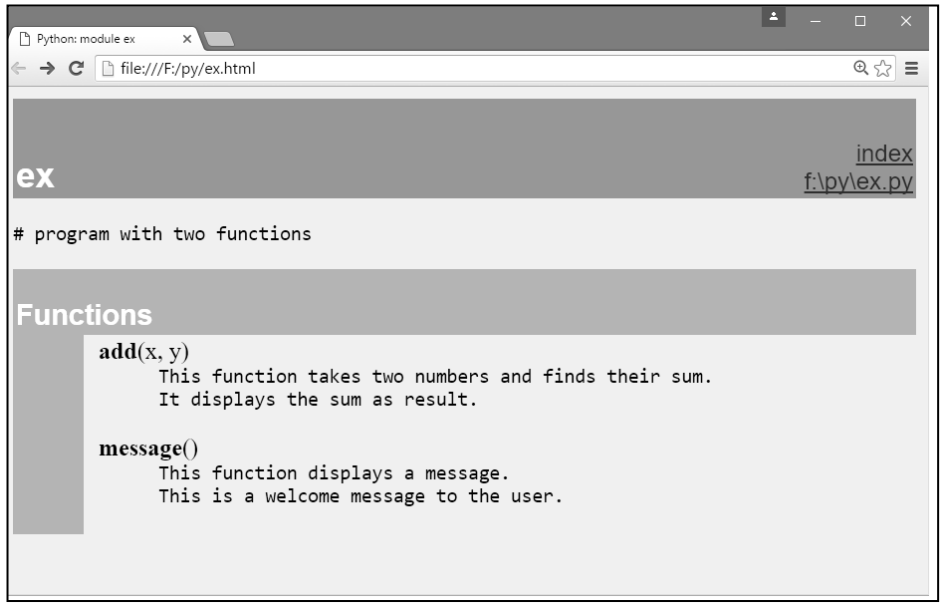


Figure 3.1: API Documentation File

After seeing Figure 3.1, we can understand that the API documentation file contains nothing but the names and the descriptions of the two functions written in our program. In this way, the API documentation file contains complete help on all the features including functions, classes, modules, etc.

After discussing about comments in Python, let's go through the following statements:

```
a = 10
b = 15
```

In the first statement, the value 10 is stored into the left side variable 'a'. Storage is done by the symbol '=' which is called *assignment operator*. In the second statement, the value 15 is stored into another variable 'b'. A variable can be imagined like a memory location where we can store data. The name given to a variable is called *identifier*. Identifiers represent names of variables, functions, objects or any such thing.

Depending upon the type of data stored into a variable, Python interpreter understands how much memory is to be allocated for that variable. Hence, we need not declare the datatype of the variable explicitly like we do in case of C or Java. For example, in C or Java we declare a variable as 'int' type as:

```
int a;    //declare a as int type variable
a = 10;  //store 10 into a
```

But, in Python we can simply write as:

```
a = 10
```

This creates a variable by the name 'a' and stores the value 10. Since we are storing 10 which is an integer, Python will understand that 'a' is of integer type variable and allocates memory required to store an integer value. On the other hand, if we store a string (a group of characters) into 'a', then Python will understand 'a' as string type variable and then allocates memory required to store the string.

How Python Sees Variables

In programming languages like C, Java and many other languages, the concept of variable is connected to memory location. In all languages, a variable is imagined as a storage box which can store some value. Suppose, we write a statement as:

```
a = 1;
```

Some memory is allocated with the name 'a' and there the value '1' is stored. Here, we can imagine the memory as a box or container that stores the value, as shown in Figure 3.2:

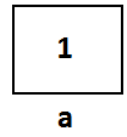


Figure 3.2: The effect of a = 1

In this way, for every variable we create, there will be a new box created with the variable name to store the value. If we change the value of the variable, then the box will be updated with the new value. For example, if we write:

```
a = 2;
```

then the new value '2' is stored into the same box, as shown in Figure 3.3:

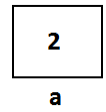


Figure 3.3: The effect of a = 2

When we assign one variable to another variable as:

```
int b = a;
```

A new memory box is created by the name 'b' and the value of the variable 'a' is copied into that box as shown in Figure 3.4:

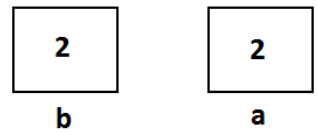


Figure 3.4: The effect of b = a;

This is how variables are visualized in other languages. However in Python, a variable is seen as a tag (or name) that is tied to some value. For example, the statement:

```
a = 1
```

means the value '1' is created first in memory and then a tag by the name 'a' is created to show that value as shown in Figure 3.5:

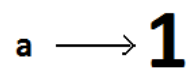


Figure 3.5: The effect of a = 1

Python considers the values (i.e. 1 or 2 etc.) as 'objects'. If we change the value of 'a' to a new value as:

```
a = 2
```

then the tag is simply changed to the new value (or object), as shown in Figure 3.6. Since the value '1' becomes unreferenced object, it is removed by garbage collector.



Figure 3.6: The effect of a = 2

Assigning one variable to another variable makes a new tag bound to the same value. For example,

```
b = a
```

Here, we are storing 'a' value into 'b'. A new tag 'b' will be created that refers to '2' as shown in Figure 3.7:



Figure 3.7: The effect of b = a

Let's understand that while other languages have variables, Python has 'tags' (or names) to represent the values. Compare Figure 3.4 and Figure 3.7. In Figure 3.4, there are two memory locations created to store two values. But in Figure 3.7, there is only one memory referenced by two names. It means Python is using memory efficiently. This is the actual way of understanding Python variables.

Datatypes in Python

A datatype represents the type of data stored into a variable or memory. The datatypes which are already available in Python language are called *Built-in* datatypes. The datatypes which can be created by the programmers are called *User-defined* datatypes.

Built-in datatypes

The built-in datatypes are of five types:

- None Type
- Numeric types
- Sequences
- Sets
- Mappings

The None Type

In Python, the 'None' datatype represents an object that does not contain any value. In languages like Java, it is called 'null' object. But in Python, it is called 'None' object.

In a Python program, maximum of only one 'None' object is provided. One of the uses of 'None' is that it is used inside a function as a default value of the arguments. When calling the function, if no value is passed, then the default value will be taken as 'None'. If

some value is passed to the function, then that value is used by the function. In Boolean expressions, 'None' datatype represents 'False'.

Numeric Types

The numeric types represent numbers. There are three sub types:

- int
- float
- complex

int Datatype

The int datatype represents an integer number. An integer number is a number without any decimal point or fraction part. For example, 200, -50, 0, 9888998700, etc. are treated as integer numbers. Now, let's store an integer number -57 into a variable 'a'.

```
a = -57
```

Here, 'a' is called int type variable since it is storing -57 which is an integer value. In Python, there is no limit for the size of an int datatype. It can store very large integer numbers conveniently.

float Datatype

The float datatype represents floating point numbers. A floating point number is a number that contains a decimal point. For example, 0.5, -3.4567, 290.08, 0.001 etc. are called floating point numbers. Let's store a float number into a variable 'num' as:

```
num = 55.67998
```

Here num is called float type variable since it is storing floating point value. Floating point numbers can also be written in scientific notation where we use 'e' or 'E' to represent the power of 10. Here 'e' or 'E' represents 'exponentiation'. For example, the number 2.5×10^4 is written as 2.5E4. Such numbers are also treated as floating point numbers. For example,

```
x = 22.55e3
```

Here, the float value 22.55×10^3 is stored into the variable 'x'. The type of the variable 'x' will be internally taken as float type. The convenience in scientific notation is that it is possible to represent very big numbers using less memory.

Complex Datatype

A complex number is a number that is written in the form of a + bj or a + bJ. Here, 'a' represents the real part of the number and 'b' represents the imaginary part of the number. The suffix 'j' or 'J' after 'b' indicates the square root value of -1. The parts 'a' and 'b' may contain integers or floats. For example, 3+5j, -1-5.5J, 0.2+10.5J are all complex numbers. See the following statement:

```
c1 = -1-5.5J
```

Here, the complex number -1-5.5J is assigned to the variable 'c1'. Hence, Python interpreter takes the datatype of the variable 'c1' as complex type.

We will write a Python program (Program 1) that adds two complex numbers and displays their sum. In this program, we store a complex number in a variable 'c1' and another complex number in another variable 'c2' and then add them. The result of addition is stored into 'c3' which is another complex type variable and then the value of 'c3' is displayed by the print() function.

Program

Program 1: A Python program to display the sum of two complex numbers.

```
# python program to add two complex numbers
c1 = 2.5+2.5J
c2 = 3.0-0.5J
c3 = c1 + c2
print("Sum= ", c3)
```

Output:

```
C:\>python add.py
Sum= (5.5+2j)
```

Representing Binary, Octal and Hexadecimal Numbers

A binary number should be written by prefixing 0b (zero and b) or 0B (zero and B) before the value. For example, 0b110110, 0B101010011 are treated as binary numbers. Hexadecimal numbers are written by prefixing 0x (zero and x) or 0X (zero and big X) before the value, as 0xA180 or 0X11fb91 etc. Similarly, octal numbers are indicated by prefixing 0o (zero and small o) or 0O (zero and then O) before the actual value. For example, 0O145 or 0o773 are octal values.

Converting the Datatypes Explicitly

Depending on the type of data, Python internally assumes the datatype for the variable. But sometimes, the programmer wants to convert one datatype into another type on his own. This is called *type conversion* or *coercion*. This is possible by mentioning the datatype with parentheses. For example, to convert a number into integer type, we can write int(num).

- int(x) is used to convert the number x into int type. See the example:

```
x = 15.56
int(x) # will display 15
```

- float(x) is used to convert x into float type. For example,

```
num = 15
float(num) # will display 15.0
```

- complex(x) is used to convert x into a complex number with real part x and imaginary part zero. For example,

```
n = 10
complex(n) # will display (10+0j)
```

- complex(x, y) is used to convert x and y into a complex number such that x will be the real part and y will be the imaginary part. For example,

```
a = 10
b = -5
complex(a, b) # will display (10-5j)
```

From the previous discussion, we can understand that int(x) is converting the datatype of x into int type and producing decimal integer number. So, we can use int(x) to convert a value from other number systems into our decimal number system. This is what we are trying to do in Program 2. In this program, we are going to convert binary, octal and hexadecimal numbers into decimal integers using type conversion.

Program

Program 2: A program to convert numbers from octal, binary and hexadecimal systems into decimal number system.

```
#  python program to convert into decimal number system
n1 = 0O17
n2 = 0B1110010
n3 = 0X1c2

n = int(n1)
print('Octal 17 = ', n)
n = int(n2)
print('Binary 1110010 = ', n)
n = int(n3)
print('Hexadecimal 1c2 = ', n)
```

Output:

```
C:\>python convert.py
Octal 17 =  15
Binary 1110010 =  114
Hexadecimal 1c2 =  450
```

There is a function in the form of int (string, base) that is useful to convert a string into a decimal integer. 'string' represents the string format of the number. It should contain integer number in string format. 'base' represents the base of the number system to be used for the string. For example, base 2 indicates binary number and base 16 indicates hexadecimal number. For example,

```
str = "1c2"   #  string str contains a hexadecimal number
n = int(str, 16)   # hence base is 16. Convert str into int
print(n)  # this will display 450
```

Program

Program 3: In this program, we are going to rewrite Program 2 using int() function to convert numbers from different number systems into decimal number system.

```
# python program to convert into decimal number system – v2.0
s1 = "17"
s2 = "1110010"
s3 = "1c2"

n = int(s1, 8)
print('Octal 17 = ', n)
n = int(s2, 2)
print('Binary 1110010 = ', n)
n = int(s3, 16)
print('Hexadecimal 1c2 = ', n)
```

Output:

```
C:\>python convert.py
Octal 17 =  15
Binary 1110010 =  114
Hexadecimal 1c2 =  450
```

We can also use functions bin() for converting a number into binary number system. The function oct() will convert a number into octal number system and the function hex() will convert a number into hexadecimal number system. See the following examples:

```
a = 10
b = bin(a)
print(b) # displays 0b1010

b = oct(a)
print(b) # displays 0o12

b = hex(a)
print(b) # displays 0xa
```

bool Datatype

The bool datatype in Python represents boolean values. There are only two boolean values True or False that can be represented by this datatype. Python internally represents True as 1 and False as 0. A blank string like "" is also represented as False. Conditions will be evaluated internally to either True or False. For example,

```
a = 10
b = 20
if(a<b): print("Hello") # displays Hello.
```

In the previous code, the condition a<b which is written after if - is evaluated to True and hence it will execute print("Hello").

```
a = 10>5  # here 'a' is treated as bool type variable
print(a) # displays True

a = 5>10
print(a) # displays False
```

```
True+True will display 2  # True is 1 and false is 0

True-False will display 1
```

Sequences in Python

Generally, a sequence represents a group of elements or items. For example, a group of integer numbers will form a sequence. There are six types of sequences in Python:

- str
- bytes
- bytearray
- list
- tuple
- range

str Datatype

In Python, str represents string datatype. A string is represented by a group of characters. Strings are enclosed in single quotes or double quotes. Both are valid.

```
str = "Welcome"  # here str is name of string type variable
str = 'Welcome'  # here str is name of string type variable
```

We can also write strings inside """ (triple double quotes) or ''' (triple single quotes) to span a group of lines including spaces.

```
str1 = """This is a book on Python which
discusses all the topics of Core Python
in a very lucid manner."""

str2 = '''This is a book on Python which
discusses all the topics of Core Python
in a very lucid manner.'''
```

The triple double quotes or triple single quotes are useful to embed a string inside another string as shown below:

```
str = """This is 'Core Python' book."""
print(str) # will display: This is 'Core Python' book.

str = '''This is "Core Python" book.'''
print(str) # will display: This is "Core Python" book.
```

The slice operator represents square brackets [and] to retrieve pieces of a string. For example, the characters in a string are counted from 0 onwards. Hence, str[0] indicates the 0th character or beginning character in the string. See the examples below:

```
s = 'Welcome to Core Python'  # this is the original string
print(s)
Welcome to Core Python
```

```
print(s[0])  # display 0th character from s
W

print(s[3:7])  # display from 3rd to 6th characters
come

print(s[11:])  # display from 11th characters onwards till end
Core Python

print(s[-1])  # display first character from the end
n
```

The repetition operator is denoted by '*' symbol and useful to repeat the string for several times. For example s * n repeats the string for n times. See the example:

```
print(s*2)
Welcome to Core PythonWelcome to Core Python
```

bytes Datatype

The bytes datatype represents a group of byte numbers just like an array does. A byte number is any positive integer from 0 to 255 (inclusive). bytes array can store numbers in the range from 0 to 255 and it cannot even store negative numbers. For example,

```
elements = [10, 20, 0, 40, 15]  # this is a list of byte numbers
x = bytes(elements)  # convert the list into bytes array
print(x[0])   # display 0th element, i.e 10
```

We cannot modify or edit any element in the bytes type array. For example, x[0] = 55 gives an error. Here we are trying to replace 0th element (i.e. 10) by 55 which is not allowed.

Now, let's write a Python program (Program 4) to create a bytes type array with a group of elements and then display the elements using a for loop. In this program, we are using a for loop as:

```
for i in x: print(i)
```

It means, if 'i' is found in the array 'x' then display i value using print() function. Remember, we need not declare the datatypes of variables in Python. Hence, we need not tell which type of variable 'i' is. In the above loop, 'i' stores each element of the array 'x' and with every element, print(i) will be executed once. Hence, print(i) displays all elements of the array 'x'. We will discuss for loop in detail in the next chapter.

Program

Program 4: A Python program to create a byte type array, read and display the elements of the array.

```
# program to understand bytes type array
# create a list of byte numbers
elements = [10, 20, 0, 40, 15]

# convert the list into bytes type array
x = bytes(elements)
```

```
# retrieve elements from x using for loop and display
for i in x: print(i)
```

Output:

```
C:\>python bytes.py
10
20
0
40
15
```

bytearray Datatype

The bytearray datatype is similar to bytes datatype. The difference is that the bytes type array cannot be modified but the bytearray type array can be modified. It means any element or all the elements of the bytearray type can be modified. To create a bytearray type array, we can use the function bytearray as:

```
elements = [10, 20, 0, 40, 15]  # this is a list of byte numbers
x = bytearray(elements)  # convert the list into bytearray type array
print(x[0])   # display 0th element, i.e 10
```

We can modify or edit the elements of the bytearray. For example, we can write:

```
x[0] = 88  # replace 0th element by 88
x[1] = 99  # replace 1st element by 99
```

We will write program to create a bytearray type array and then modify the first two elements. Then we will display all the elements using a for loop. This can be seen in Program 5.

Program

Program 5: A Python program to create a bytearray type array and retrieve elements

```
# program to understand bytearray type array
# create a list of byte numbers
elements = [10, 20, 0, 40, 15]

# convert the list into bytearray type array
x = bytearray(elements)

# modify the first two elements of x
x[0] = 88
x[1] = 99

# retrieve elements from x using for loop and display
for i in x: print(i)
```

Output:

```
C:\>python bytearray.py
88
99
0
40
15
```

list Datatype

Lists in Python are similar to arrays in C or Java. A list represents a group of elements. The main difference between a list and an array is that a list can store different types of elements but an array can store only one type of elements. Also, lists can grow dynamically in memory. But the size of arrays is fixed and they cannot grow at runtime. Lists are represented using square brackets [ ] and the elements are written in [ ], separated by commas. For example,

```
list = [10, -20, 15.5, 'Vijay', "Mary"]
```

will create a list with different types of elements. The slicing operation like [0: 3] represents elements from 0th to 2nd positions, i.e. 10, 20, 15.5. Have a look at some of the operations on a list in the following IDLE window, as shown in Figure 3.8:

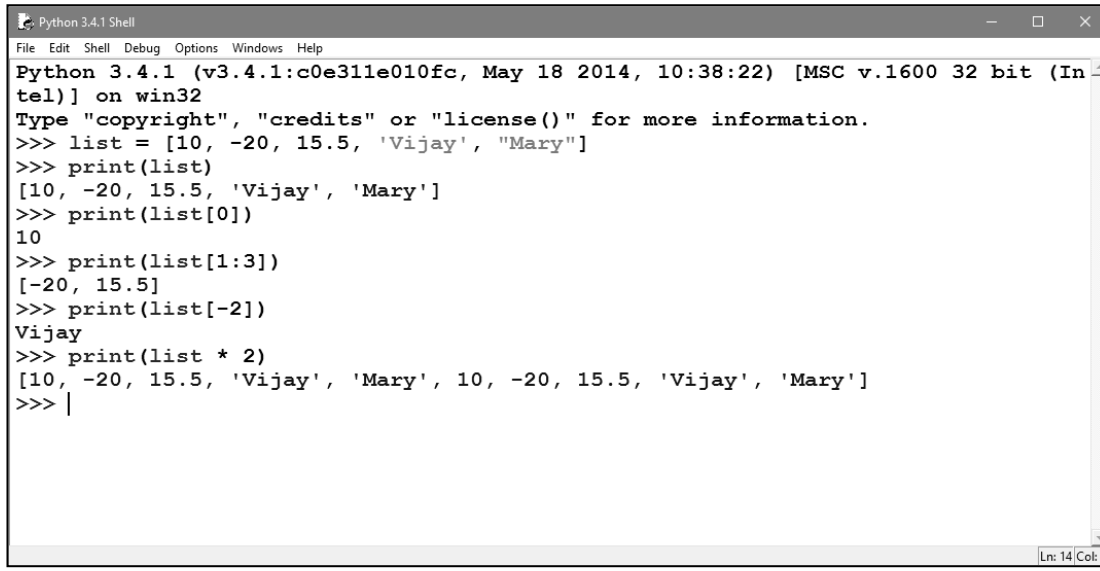


Figure 3.8: Some Operations on Lists

tuple Datatype

A tuple is similar to a list. A tuple contains a group of elements which can be of different types. The elements in the tuple are separated by commas and enclosed in parentheses ( ). Whereas the list elements can be modified, it is not possible to modify the tuple elements. That means a tuple can be treated as a read-only list. Let's create a tuple as:

```
tpl = (10, -20, 15.5, 'Vijay', "Mary")
```

The individual elements of the tuple can be referenced using square braces as tpl[0], tpl[1], tpl[2], … Now, if we try to modify the 0th element as:

```
tpl[0] = 99
```

This will result in error. The slicing operations which can be done on lists are also valid in tuples. See the general operations on the tuple in Figure 3.9:

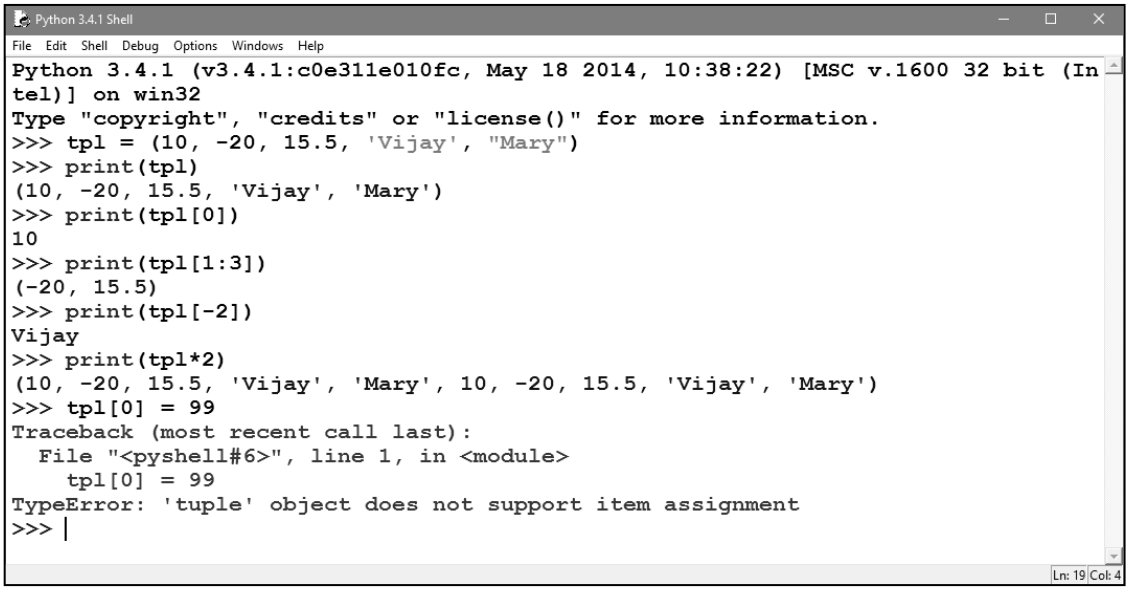


Figure 3.9: Some operations on tuples

range Datatype

The range datatype represents a sequence of numbers. The numbers in the range are not modifiable. Generally, range is used for repeating a for loop for a specific number of times. To create a range of numbers, we can simply write:

```
r = range(10)
```

Here, the range object is created with the numbers starting from 0 to 9. We can display these numbers using a for loop as:

```
for i in r: print(i)
```

The above statement will display numbers from 0 to 9. We can use a starting number, an ending number and a step value in the range object as:

```
r = range(30, 40, 2)
```

This will create a range object with a starting number 30 and an ending number 39. The step size is 2. It means the numbers in the range will increase by 2 every time. So, the for loop

```
for i in r: print(i)
```

will display numbers: 30, 32, 34, 36, 38. Let's create a list with a range of numbers from 0 to 9 as:

```
lst = list(range(10))
print(lst) # will display: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Sets

A set is an unordered collection of elements much like a set in Mathematics. The order of elements is not maintained in the sets. It means the elements may not appear in the same order as they are entered into the set. Moreover, a set does not accept duplicate elements. There are two sub types in sets:

- set datatype
- frozenset datatype

set Datatype

To create a set, we should enter the elements separated by commas inside curly braces {}.

```
s = {10, 20, 30, 20, 50}
print(s) # may display {50, 10, 20, 30}
```

Please observe that the set 's' is not maintaining the order of the elements. We entered the elements in the order 10, 20, 30, 20 and 50. But it is showing another order. Also, we repeated the element 20 in the set, but it has stored only one 20. We can use the set() function to create a set as:

```
ch = set("Hello")
print(ch) # may display {'H', 'e', 'l', 'o'}
```

Here, a set 'ch' is created with the characters H,e,l,o. Since a set does not store duplicate elements, it will not store the second 'l'. We can convert a list into a set using the set() function as:

```
lst = [1,2,5,4,3]
s = set(lst)
print(s) # may display {1, 2, 3, 4, 5}
```

Since sets are unordered, we cannot retrieve the elements using indexing or slicing operations. For example, the following statements will give error messages:

```
print(s[0])   # indexing, display 0th element
print(s[0:2])  # slicing, display from 0 to 1st elements
```

The update() method is used to add elements to a set as:

```
s.update([50,60])
print(s) # may display {1, 2, 3, 4, 5, 50, 60}
```

On the other hand, the remove() method is used to remove any particular element from a set as:

```
s.remove(50)
print(s) # may display {1, 2, 3, 4, 5, 60}
```

frozenset Datatype

The frozenset datatype is same as the set datatype. The main difference is that the elements in the set datatype can be modified; whereas, the elements of frozenset cannot be modified. We can create a frozenset by passing a set to frozenset() function as:

```
s = {50,60,70,80,90}
print(s) # may display {80, 90, 50, 60, 70}

fs = frozenset(s)  # create frozenset fs
print(fs) # may display frozenset({80, 90, 50, 60, 70})
```

Another way of creating a frozenset is by passing a string (a group of characters) to the frozenset() function as:

```
fs = frozenset("abcdefg")
print(fs) # may display frozenset({'e', 'g', 'f', 'd', 'a', 'c', 'b'})
```

However, update() and remove() methods will not work on frozensets since they cannot be modified or updated.

Mapping Types

A map represents a group of elements in the form of key value pairs so that when the key is given, we can retrieve the value associated with it. The *dict* datatype is an example for a map. The 'dict' represents a 'dictionary' that contains pairs of elements such that the first element represents the key and the next one becomes its value. The key and its value should be separated by a colon (:) and every pair should be separated by a comma. All the elements should be enclosed inside curly brackets { }.

We can create a dictionary by typing the roll numbers and names of students. Here, roll numbers are keys and names will become values. We write these key value pairs inside curly braces as:

```
d = {10: 'Kamal', 11: 'Pranav', 12:  'Hasini', 13: 'Anup', 14:
'Reethu'}
```

Here, d is the name of the dictionary. 10 is the key and its associated value is 'Kamal'. The next key is 11 and its value is 'Pranav'. Similarly 12 is the key and 'Hasini' is it value. 13 is the key and 'Anup' is the value and 14 is the key and 'Reethu' is the value.

We can create an empty dictionary without any elements as:

```
d = {}
```

Later, we can store the key and values into d as:

```
d[10] = 'Kamal'
d[11] = 'Pranav'
```

In the preceding statements, 10 represents the key and 'Kamal' is its value. Similarly, 11 represents the key and its value is 'Pranav'. Now, if we write

```
print(d)
```

3. Multiplication, division, modulus and floor divisions are at equal priority. d = 27//2+3 and then d = 13+3.
4. Addition and subtraction are done afterwards. d = 16.
5. Finally, assignment is performed. The value 16 is now stored into 'd'.

Hence, the total value of the expression becomes 16 which is stored in the variable 'd'.

Using Python Interpreter as Calculator

It is interesting to know that we can use Python interpreter as a simple calculator that can perform basic arithmetic calculations. Open Python IDLE graphics window and type some arithmetic operations, as shown in Figure 4.1:

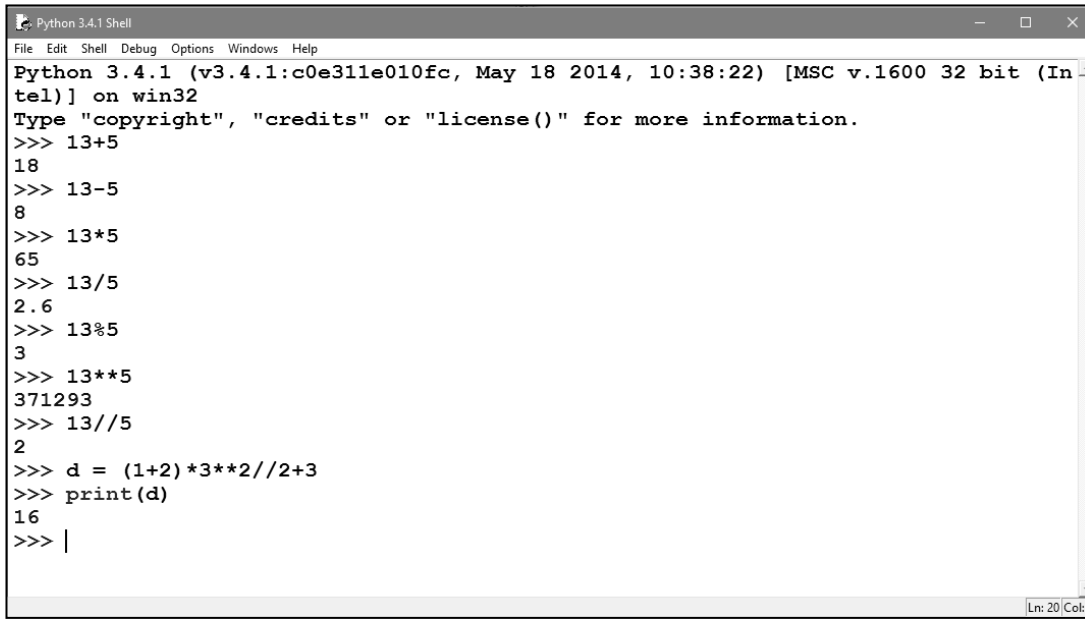


Figure 4.1: Python Interpreter as Calculator

Assignment Operators

These operators are useful to store the right side value into a left side variable. They can also be used to perform simple arithmetic operations like addition, subtraction, etc., and then store the result into a variable. These operators are shown in Table 4.2. In Table 4.2, let's assume the values x = 20, y = 10 and z = 5:

Table 4.2: Assignment Operators

Operator	Example	Meaning	Result
=	z = x+y	Assignment operator. Stores right side value into left side variable, i.e. x+y is stored into z.	z = 30
+=	z+=x	Addition assignment operator. Adds right operand to the left operand and stores the result into left operand, i.e. z = z+x.	z = 25
-=	z-=x	Subtraction assignment operator. Subtracts right operand from left operand and stores the result into left operand, i.e. z = z-x.	z = -15
*=	z*=x	Multiplication assignment operator. Multiplies right operand with left operand and stores the result into left operand, i.e. z = z *x.	z = 100
/=	z/=x	Division assignment operator. Divides left operand with right operand and stores the result into left operand, i.e. z = z/x.	z = 0.25
%=	z%=x	Modulus assignment operator. Divides left operand with right operand and stores the remainder into left operand, i.e. z = z%x.	z = 5
**=	z**=y	Exponentiation assignment operator. Performs power value and then stores the result into left operand, i.e. z = z**y.	z= 9765625
//=	z//=y	Floor division assignment operator. Performs floor division and then stores the result into left operand, i.e. z = z// y.	z = 0

It is possible to assign the same value to two variables in the same statement as:

```
a=b=1
print(a, b) # will display 1 1
```

Another example is where we can assign different values to two variables as:

```
a=1; b=2
print(a, b) # will display 1 2
```

The same can be done using the following statement:

```
a, b = 1, 2
print(a, b) # will display 1 2
```

A word of caution: Python does not have increment operator (++) and decrement operator (--) that are available in C and Java.

Unary Minus Operator

The unary minus operator is denoted by the symbol minus (-). When this operator is used before a variable, its value is negated. That means if the variable value is positive, it will be converted into negative and vice versa. For example, consider the following statements:

```
n = 10
print(-n) # displays -10

num = -10
num = - num
print(num) # displays 10
```

Relational Operators

Relational operators are used to compare two quantities. We can understand whether two values are same or which one is bigger or which one is lesser, etc. using these operators. These operators will result in True or False depending on the values compared, as shown in Table 4.3. In this table, we are assuming a =1 and b = 2.

Table 4.3: Relational Operators

Operator	Example	Meaning	Result
>	a>b	Greater than operator. If the value of left operand is greater than the value of right operand, it gives True else it gives False.	False
>=	a>=b	Greater than or equal operator. If the value of left operand is greater or equal than that of right operand, it gives True else False.	False
<	a<b	Less than operator. If the value of left operand is less than the value of right operand, it gives True else it gives False.	True
<=	a<=b	Less than or equal operator. If the value of left operand is lesser or equal than that of right operand, it gives True else False.	True
==	a==b	Equals operator. If the value of left operand is equal to the value of right operand, it gives True else False.	False
!=	a != b	Not equals operator. If the value of the left operand is not equal to the value of right operand, it returns True else it returns False.	True

Relational operators are generally used to construct conditions in if statements. For example,

```
a=1; b=2
if (a>b):
    print("Yes")
else:
    print("No")
```

will display 'No'. Observe the expression (a>b) written after if. This is called a 'condition'. When this condition becomes True, if statement will display 'Yes' and if it becomes False, then it will display 'No'. In this case, (1>2) is False and hence 'No' will be displayed.

Relational operators can be chained. It means, a single expression can hold more than one relational operator. For example,

```
x=15
10<x<20  # displays True
```

Here, 10 is less than 15 is True, and then 15 is less than 20 is True. Since both the conditions are evaluated to True, the result will be True.

```
10>=x<20 # displays False
```

Here, 10 is greater than or equal to 15 is False. But 15 is less than 20 is True. Since we get False and True, the result will be False.

```
10<x>20 # displays False
```

Here, 10 is less than 15 is True. But 15 is greater than 20 is False. Since we are getting True and False, the total result will be False. So, the point is this: in the chain of relational operators, if we get all True, then only the final result will be True. If any comparison yields False, then we get False as the final result. Thus,

```
1<2<3<4 # will give True

1<2>3<4 # will give False

4>2>=2>1 # will give True
```

Logical Operators

Logical operators are useful to construct compound conditions. A compound condition is a combination of more than one simple condition. Each of the simple condition is evaluated to True or False and then the decision is taken to know whether the total condition is True or False. We should keep in mind that in case of logical operators, False indicates 0 and True indicates any other number. There are 3 logical operators as shown in Table 4.4. Let's take x = 1 and y=2 in this table.

Table 4.4: Logical Operators

Operator	Example	Meaning	Result
And	x and y	And operator. If x is False, it returns x, otherwise it returns y.	2
Or	x or y	Or operator. If x is False, it returns y, otherwise it returns x.	1
Not	not x	Not operator. If x is False, it returns True, otherwise True.	False

Let's take some statements to understand the effect of logical operators.

```
x = 100
y = 200
print(x and y)  # will display 200

print(x or y)  # will display 100

print (not x) # will display False

x=1;  y=2; z=3
if(x<y and y<z): print('Yes')
else: print('No')
```

In the above statement, observe the compound condition after if. That is x <y and y<z. This is a combination of two simple conditions, x<y and y<z. When 'and' is used, the total condition will become True only if both the conditions are True. Since both the conditions became True, we will get 'Yes' as output. Observe another statement below:

```
if(x>y or y<z): print('Yes')
else: print('No')
```

Here, x>y is False but y<z is True. When using 'or' if any one condition is True, it will take the total compound condition as 'True' and hence it will display 'Yes'.

Boolean Operators

We know that there are two 'bool' type literals. They are True and False. Boolean operators act upon 'bool' type literals and they provide 'bool' type output. It means the result provided by Boolean operators will be again either True or False. There are three Boolean operators as mentioned in Table 4.5. Let's take x = True and y = False in Table 4.5:

Table 4.5: Boolean Operators

Operator	Example	Meaning	Result
and	x and y	Boolean and operator. If both x and y are True, then it returns True, otherwise False.	False
or	x or y	Boolean or operator. If either x or y is True, then it returns True, else False.	True
not	not x	Boolean not operator. If x is True, it returns False, else True.	False

In Figure 4.2, we can observe the effect of Boolean operators:

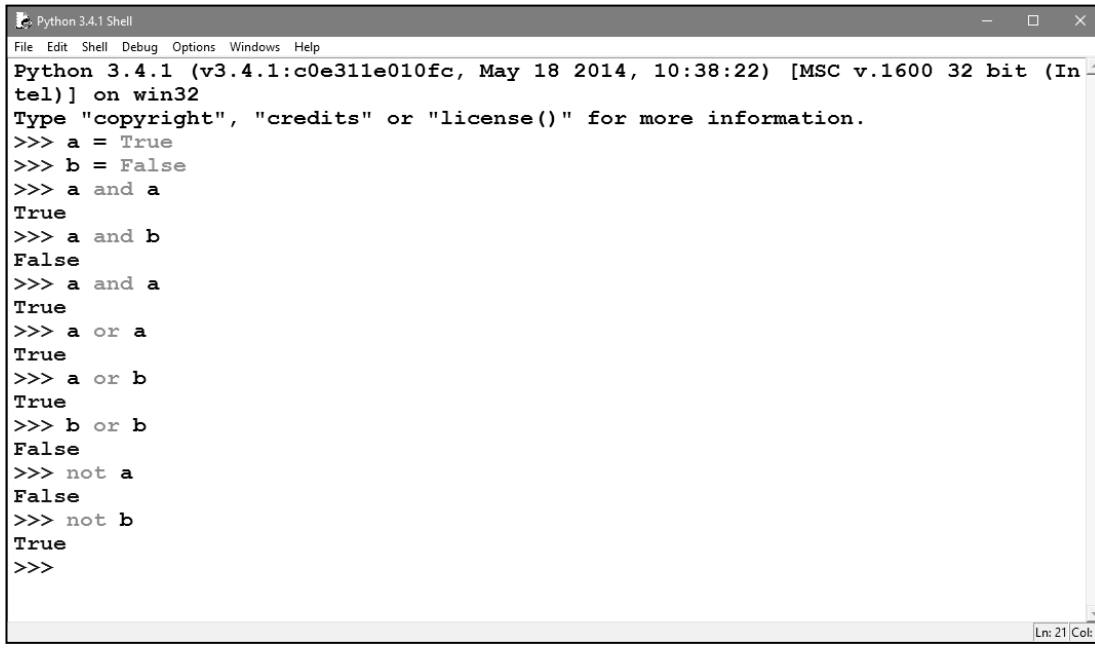


Figure 4.2: Boolean Operators and Their Usage

Bitwise Operators

These operators act on individual bits (0 and 1) of the operands. We can use bitwise operators directly on binary numbers or on integers also. When we use these operators on integers, these numbers are converted into bits (binary number system) and then bitwise operators act upon those bits. The results given by these operators are always in the form of integers.

We use decimal number system in our daily life. This number system consists of 10 digits from 0 to 9. We count all numbers using these 10 digits only. But in case of binary number system that is used by computers internally, there are only 2 digits, i.e. 0 and 1 which are called *bits* (binary digits). All values are represented only using these two bits. It is possible to convert a decimal number into binary number and vice versa.

Example 1: Converting 45 into binary number system.

Rule: Divide the number successively by 2 and take the remainders from bottom to top, as shown in Figure 4.3. The decimal number 45 is represented as 101101 in binary. If we use 8 bit representation, we can write it as: 0010 1101.

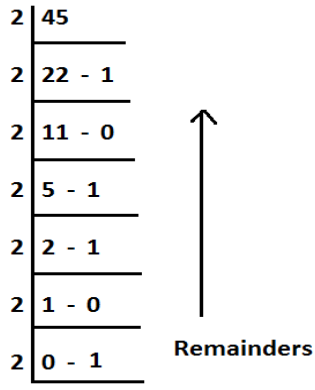


Figure 4.3: Converting into Binary

Example 2: Converting binary number 0010 1101 into decimal number.

Rule: Multiply the individual bits by the powers of 2 and take the sum of the products, as shown in Figure 4.4. Here, the sum is coming to 45. So 0010 1101 in binary is equal to 45 in decimal number system.

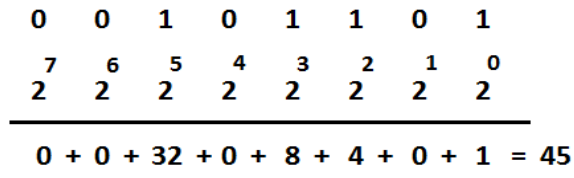


Figure 4.4: Converting from Binary to Decimal

There are 6 types of bitwise operators as shown below:

- Bitwise Complement operator (~)
- Bitwise AND operator (&)
- Bitwise OR operator (|)
- Bitwise XOR operator (^)
- Bitwise Left shift operator (<<)
- Bitwise Right shift operator (>>)

We will discuss each of these operators one by one.

Bitwise Complement Operator (~)

This operator gives the complement form of a given number. This operator symbol is ~, which is pronounced as *tilde*. Complement form of a positive number can be obtained by changing 0's as 1's and vice versa. The complement operation is performed by NOT gate circuit in electronics. Truth table is a table that gives relationship between the inputs and the output. The truth table is also given for NOT gate, as shown in Figure 4.5:

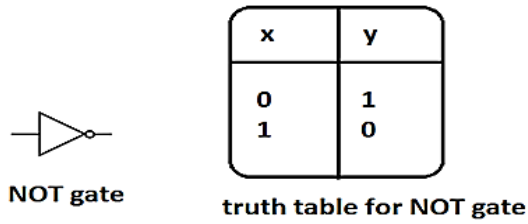


Figure 4.5: NOT gate that Performs Complement of Bits

```
If x = 10, find the ~x value.
x = 10 = 0000 1010.
```

By changing 0's as 1's and vice versa, we get 1111 0101. This is nothing but -11(in decimal). So, ~x = -11.

Bitwise AND Operator (&)

This operator performs AND operation on the individual bits of numbers. The symbol for this operator is &, which is called *ampersand*. To understand the bitwise AND operation, see the truth table given in Figure 4.6:

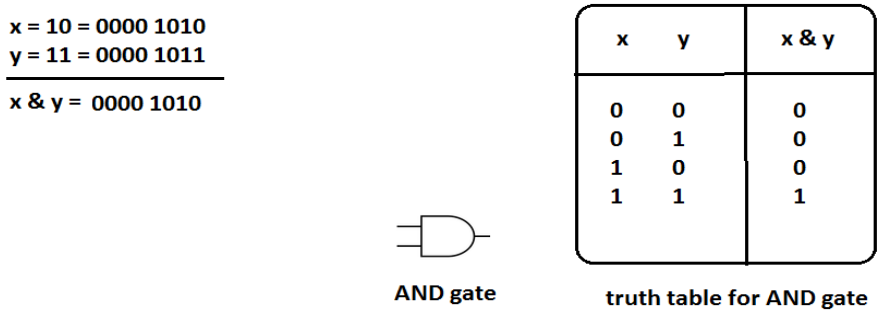


Figure 4.6: AND Operation

From the truth table, we can conclude that by multiplying the input bits, we can get the output bit. The AND gate circuit present in the computer chip will perform the AND operation.

```
If x = 10, y = 11. Find the value of x&y.
x = 10 = 0000 1010.
y = 11 = 0000 1011.
```

From the truth table, by multiplying the bits, we can get x&y = 0000 1010. This is nothing but 10 (in decimal).

Bitwise OR Operator (|)

This operator performs OR operation on the bits of the numbers. The symbol of bitwise OR operator is |, which is called *pipe* symbol. To understand this operation, see the truth table given in Figure 4.7. From the table, we can conclude that by adding the input bits, we can get the output bit. The OR gate circuit, which is present in the computer chip will perform the OR operation.

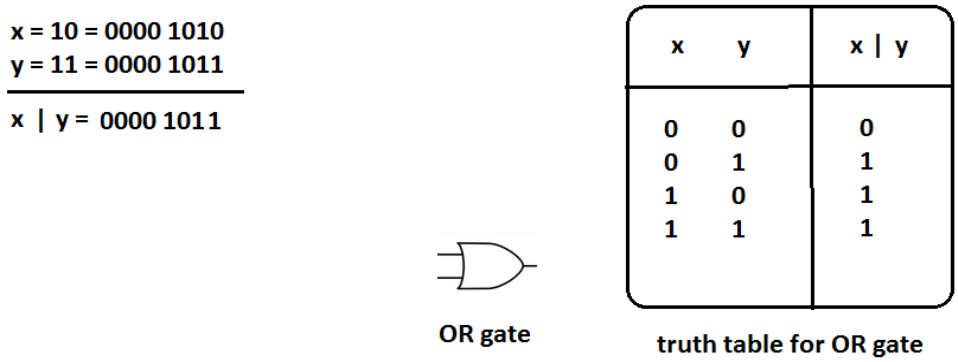


Figure 4.7: OR Operation

```
If x = 10, y = 11, find the value of x|y.
x = 10 = 0000 1010.
y = 11 = 0000 1011.
```

The truth table shows that by adding the bits, we can get x|y = 0000 1011. This is nothing but 11 (in decimal).

Bitwise XOR Operator (^)

This operator performs *exclusive or (XOR)* operation on the bits of numbers. The symbol is ^, which is called *cap*, *carat*, or *circumflex* symbol. To understand the *XOR* operation, see the truth table given in Figure 4.8. From the table, we can conclude that when we have odd number of 1's in the input bits, we can get the output bit as 1. The *XOR* gate circuit of the computer chip will perform this operation.

x = 10 = 0000 1010
y = 11 = 0000 1011

x ^ y = 0000 0001



XOR gate

x	y	x ^ y
0	0	0
0	1	1
1	0	1
1	1	0

truth table for XOR gate

Figure 4.8: XOR Operation

```
If x = 10, y = 11, find the value of x^y.
x = 10 = 0000 1010.
y = 11 = 0000 1011.
```

From the truth table, when odd number of 1's are there, we can get a 1 in the output. Thus, x^y = 0000 0001 is nothing but 1 (in decimal).

Bitwise Left Shift Operator (<<)

This operator shifts the bits of the number towards left a specified number of positions. The symbol for this operator is <<, read as *double less than*. If we write x<<n, the meaning is to shift the bits of x towards left n positions.

```
If x = 10, calculate x value if we write x<<2.
```

Shifting the value of x towards left 2 positions will make the leftmost 2 bits to be lost. The value of x is 10 = 0000 1010. Now, x<<2 will be 0010 1000 = 40 (in decimal). The procedure to do this is explained, as shown in Figure 4.9:

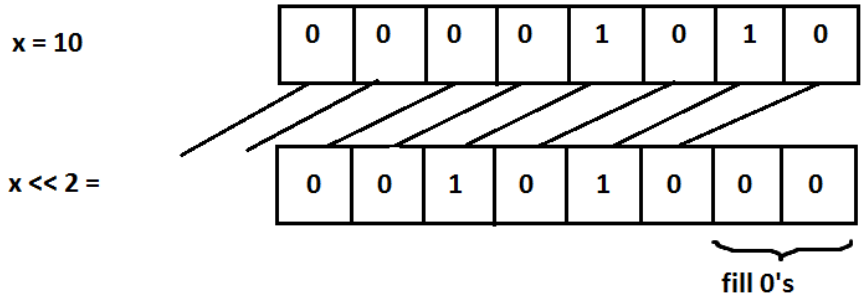


Figure 4.9: Shifting bits towards left 2 times

Bitwise Right Shift Operator (>>)

This operator shifts the bits of the number towards right a specified number of positions. The symbol for this operator is >>, read as *double greater than*. If we write x>>n, the meaning is to shift the bits of x towards right n positions.

>> shifts the bits towards right and also preserves the sign bit, which is the leftmost bit. Sign bit represents the sign of the number. Sign bit 0 represents a positive number and 1 represents a negative number. So, after performing >> operation on a positive number, we get a positive value in the result also. If right shifting is done on a negative number, again we get a negative value only.

```
If x = 10, then calculate x>>2 value.
```

Shifting the value of x towards right 2 positions will make the rightmost 2 bits to be lost. x value is 10 = 0000 1010. Now x>>2 will be: 0000 0010 = 2 (in decimal) (Figure 4.10).

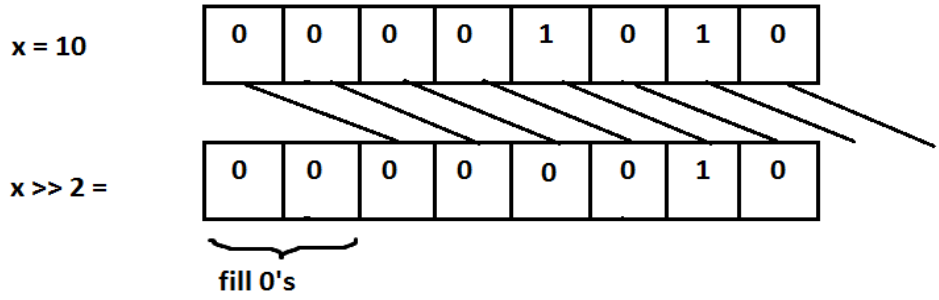


Figure 4.10: Shifting bits towards right 2 times

Let's check the effects of various bitwise operators so far discussed. We will use IDLE window and type the Python statements and confirm the results, as shown in Figure 4.11:

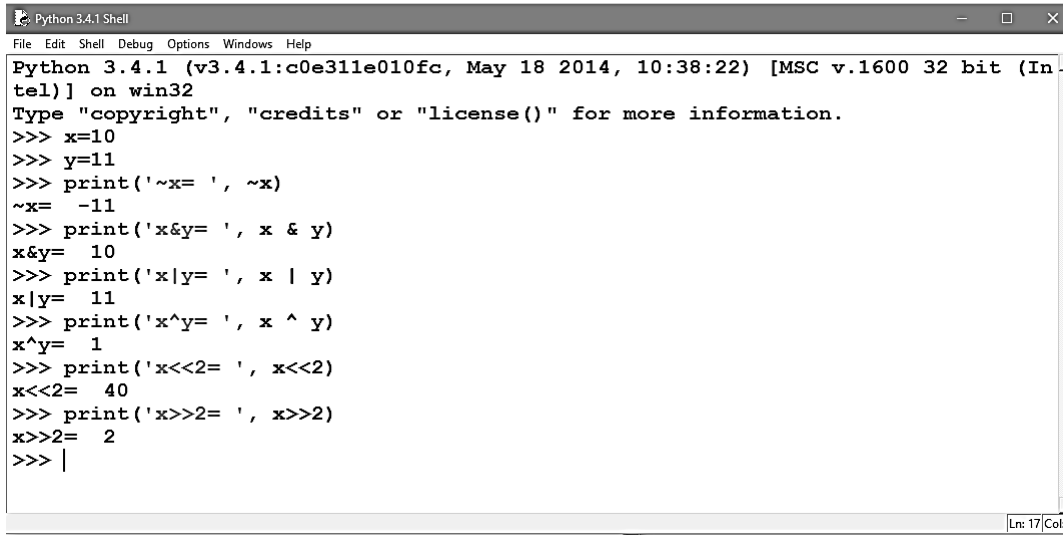


Figure 4.11: Using the Python Bitwise Operators

Membership Operators

The membership operators are useful to test for membership in a sequence such as strings, lists, tuples or dictionaries. For example, if an element is found in the sequence or not can be asserted using these operators. There are two membership operators as shown here:

- in
- not in

The in Operator

This operator returns True if an element is found in the specified sequence. If the element is not found in the sequence, then it returns False.

The not in Operator

This works in reverse manner for 'in' operator. This operator returns True if an element is not found in the sequence. If the element is found, then it returns False.

Let's take a group of strings in a list. We want to display the members of the list using a for loop where the 'in' operator is used. The list of names is given below:

```
names = ["Rani", "Yamini", "Sushmita", "Veena"]
```

Here the list name is 'names'. It contains a group of names. Suppose, we want to retrieve all the names from this list, we can use a for loop as:

```
for name in names:
    print (name)
```

Output:

```
Rani
Yamini
Sushmita
Veena
```

In the for loop, the variable 'name' is used. This variable will store each and every value of the list 'names'. It means, 'name' values will be "Rani", "Yamini", etc. The operator 'in' will check whether each of these names is a member of the list or not. For example, "Rani" is a member of the list and hence, 'in' will return True. When 'True' is returned, the print() function in the for loop is executed and that name is displayed. In this way, all the names of the list are displayed.

Let's take another example where we want to display the elements of a dictionary. The dictionary 'postal' contains the names of the cities and their pin codes. The city name becomes a key and the pin code will become its value. For example,

```
postal = {'Delhi': 110001, 'Chennai': 600001, 'Kolkata': 700001,
'Bangalore':560001}
```

Now, we want to retrieve each key and its corresponding value. For this purpose, we take a variable 'city' in for loop. Every time in the for loop, 'city' will get the name of the city. To get the corresponding value, we can use postal[city]. So, the following for loop will display the cities and their pin codes from 'postal' dictionary:

```
for city in postal:
    print(city, postal[city])
```

Output:

```
Kolkata 700001
Bangalore 560001
Delhi 110001
Chennai 600001
```

Identity Operators

These operators compare the memory locations of two objects. Hence, it is possible to know whether the two objects are same or not. The memory location of an object can be seen using the id() function. This function returns an integer number, called the *identity number* that internally represents the memory location of the object. For example, id(a) gives the identity number of the object referred by the name 'a'. See the following statements:

```
a = 25
b = 25
```

In the first statement, we are assigning the name (or identifier) 'a' to the object 25. In the second statement, we are assigning another name 'b' to the same object 25. In Python, everything is considered as an object. Here, 25 is the object for which two names are given. If we display an identity number of these two variables, we will get same numbers as they refer to the same object.

```
id(a)
1670954952
id(b)
1670954952
```

There are two identity operators:

- is
- is not

The is Operator

The 'is' operator is useful to compare whether two objects are same or not. It will internally compare the identity number of the objects. If the identity numbers of the objects are same, it will return True; otherwise, it returns False.

The is not Operator

This is not operator returns True, if the identity numbers of two objects being compared are not same. If they are same, then it will return False.

The 'is' and 'is not' operators do not compare the values of the objects. They compare the identity numbers or memory locations of the objects. If we want to compare the value of the objects, we should use equality operator (==).

```
a = 25
b = 25
if(a is b):
    print("a and b have same identity")
else:
    print("a and b do not have same identity")
```

Output:

```
a and b have same identity
```

As another example, we will take two lists with 4 elements each as:

```
one = [1,2,3,4]
two = [1,2,3,4]
if(one is two):
    print("one and two are same")
else:
    print("one and two are not same")
```

Output:

```
one and two are not same
```

In the preceding example, the lists one and two are having same elements or values. But the output is "one and two are not same". The reason is this: 'is' operator does not compare the values. It compares the identity numbers of the lists. Let's see the identity numbers of these two lists by using the id() function:

```
id(one)
51792432
id(two)
51607192
```

Since both the lists are created at different memory locations, we have their identity numbers different. So, 'is' operator will take the lists 'one' and 'two' as two different lists even though their values are same. It means, 'is' is not comparing their values. We can use equality (==) operator to compare their values as:

```
if(one == two):
    print("one and two are same")
else:
    print("one and two are not same")
```

Output:

```
one and two are same
```

Operator Precedence and Associativity

An expression or formula may contain several operators. In such a case, the programmer should know which operator is executed first and which one is executed next. The sequence of execution of the operators is called *operator precedence*. The following table summarizes the operators according to their precedence. The table shows precedence in descending order. It means the operators which are having highest precedence are listed at the top of the table, shown in Table 4.6:

Table 4.6: Precedence of Operators in Python

Operator	Name
( )	Parenthesis
**	Exponentiation
-, ~	Unary minus, Bitwise complement
*, / , // , %	Multiplication, Division, Floor division, Modulus
+ , -	Addition, Subtraction
<< , >>	Bitwise left shift, Bitwise right shift
&	Bitwise AND
^	Bitwise XOR
|	Bitwise OR
>, >=, <, <= , == , !=	Relational (comparison) operators
= , %= , /= , //= , -= , += , *= , **=	Assignment operators
is , is not	Identity operators
in , not in	Membership operators
not	Logical not
or	Logical or
and	Logical and

Precedence represents the priority level of the operator. The operators with higher precedence will be executed first than of lower precedence.